ShelfLife

Smart Inventory, Billing, Expiry & Food Waste Management System

Smart Inventory. Less Waste.

University / Department	University of Karachi, UBIT
Course	Object-Oriented Programming (OOP), Second Semester
Supervisor	Miss Ilsa
Team	Five-member group project
Language / GUI / IDE	Java / Java Swing / NetBeans
Storage	Local text/CSV files (file handling)

1. Overview

ShelfLife is a Java desktop application for small food businesses. It tracks stock **by batch**, monitors expiry dates, recommends stock using **FEFO (First-Expired, First-Out)**, connects POS billing to inventory, and records food waste with its cost. This is a proposal: it describes intended design and does not claim anything is implemented or tested yet.

2. Problem Statement

- Stock is often tracked per product, hiding batch-level expiry dates.
- Expiry warnings are missed when records are manual or disconnected.
- Billing and stock sheets drift apart and become inconsistent.
- Businesses cannot see the quantity or cost of discarded goods.
- Small businesses cannot afford complex enterprise systems.

3. Aim and Objectives

Aim: Build a Java desktop system that tracks inventory by batch, applies FEFO, links billing to stock, and reports waste.

- Maintain product, supplier and batch records with unique IDs.
- Show low-stock, near-expiry and expired-stock alerts.
- Allocate sales to eligible batches in FEFO order.
- Deduct stock only after a successful checkout commit.
- Record payments and outstanding balances.
- Track waste quantity, reason and cost.
- Produce sales, inventory and waste reports with CSV export.
- Demonstrate OOP principles and reliable file handling.

4. Scope

In scope (MVP):

- Single business, local desktop use, fully offline
- Product, batch, supplier and purchase-receiving management
- Expiry alerts, FEFO allocation, stock movement history
- POS cart, invoice, payment record, receipt preview
- Waste records, dashboard, core reports, CSV export, backup/restore
- Input validation and failed-write recovery

Out of scope (future): payment gateways, cloud sync, multi-branch, SaaS, AI forecasting, tax-system integration, multi-computer editing of the same files.

5. MVP Features

Module	Key features
Products & Batches	Add/edit/search products; categories and units; multiple batches per product with own expiry, quantity and unit cost
Expiry & FEFO	Configurable alert windows; expired batches blocked from sale; earliest-expiry-first allocation with stable tie-breaker
POS & Billing	Cart, automatic pricing, invoice, stock deduction on commit, batch allocations stored per sale line, payment recording, receipt
Waste	Record batch, quantity, reason; cost = quantity x batch unit cost; waste summaries
Purchases & Suppliers	Supplier records; receiving creates traceable batches
Reports	Sales, inventory, expiry and waste reports; charts; CSV export
Reliability	Unique IDs, validation, stock movement log, dated backups, restore with validation

Phase 2 (not in MVP): roles and permissions, audit trail, discounts, partial payments, returns, cancellations, expenses, profit reports, restock suggestions, barcode support.

6. Critical Business Rules

- **Cart vs stock:** editing or abandoning a cart never changes permanent stock.
- **Checkout:** validate all lines and eligible stock first; commit invoice, allocations, movements and payment together.
- **FEFO:** deduct from the earliest-expiring eligible batch first.
- **Expired stock:** excluded from ordinary sales.
- **Failure:** a sale is never shown as complete unless all records are committed or recovered.
- **Data types:** use LocalDate for dates and BigDecimal for money.
- **Units:** never mix incompatible units silently.

Key formulas

- Waste cost = discarded quantity x batch unit cost
- Line total = quantity x selling price (less permitted discount)
- Outstanding balance = invoice total - payments received
- Gross profit = net sales - cost of goods sold

FEFO example: B001 has 5 L (expires 23 Oct) and B002 has 10 L (expires 27 Oct). A 7 L sale takes 5 L from B001 and 2 L from B002, leaving 0 L and 8 L. If only 6 L were eligible, checkout is rejected.

7. Object-Oriented Design

Main classes: Product, Batch, Supplier, StockMovement, WasteRecord, Sale, SaleItem, BatchAllocation, Payment, User.

Services: InventoryService, FEFOService, BillingService, ExpiryService, WasteService, ReportService.

Persistence: FileRepository (interface with CSV implementation).

Principle	Use in ShelfLife
Encapsulation	Private fields; batch quantity changed only through validated methods
Abstraction	GUI talks to services and repository interfaces, not file formats
Inheritance	Admin and Staff extend User only if permissions differ
Polymorphism	Shared interfaces for report/export strategies
Composition	Sale contains SaleItems; each SaleItem contains BatchAllocations
Exceptions	File and validation errors converted to clear user messages

8. Architecture and Data Design

Layers: Swing UI > Business services > Repositories > Local CSV files and backups.

Data files: products, batches, suppliers, stock_movements, sales, sale_items, batch_allocations, payments, waste_records, purchases (users optional).

Data rules:

- Dates stored as YYYY-MM-DD; IDs never depend on row numbers.
- Original unit cost saved on each batch allocation to preserve historical cost.
- Stock movements act as the audit record for quantity changes.
- Safe writes (temp file then replace) plus a transaction journal to recover from interrupted checkouts.
- CSV fields with commas or quotes must be escaped correctly.
- Backup taken before any restore; records validated afterwards.

Limitation: text files lack database transactions, so a database (e.g. SQLite) is recommended for any multi-user version.

9. Testing Plan

Test	Expected result
Duplicate ID	Save rejected, existing record unchanged
Zero/negative quantity	Validation blocks the operation
Two batches of one product	Each keeps its own expiry and balance
FEFO sale (7 L over 5 L + 10 L)	5 L from earlier batch, 2 L from later
Expired batch sale	Batch excluded; checkout rejected if stock insufficient
Insufficient stock	Checkout blocked, no invoice created
Cart edit	Permanent stock unchanged
Successful checkout	Invoice, allocations, movements and payment all match
Simulated write failure	Failure reported; incomplete sale detected or recovered
Payment of Rs 1,500 on Rs 2,000	Outstanding balance Rs 500
Waste of 2 units at Rs 100	Waste cost Rs 200
Backup and restore	Restored data validates and reconciles
Malformed CSV row	Reported, not silently misread
Expiry date boundary	Follows documented same-day policy

Exit criteria: all checkout, FEFO and stock tests pass; no test allows negative stock or sale of expired stock; reports match source data; limitations disclosed honestly.

10. Team Plan (5 Members)

Member	Responsibility
1	Product and batch models, product screens, validation, basic access
2	Stock receiving, movements, expiry alerts, FEFO logic and tests
3	POS cart, pricing, invoice, payment records, receipt
4	Waste module, dashboard, charts, CSV export, report verification
5	Repositories, navigation, backup/recovery, integration testing

Team rules: agree class names, interfaces and file formats first; use Git with small reviewable changes; keep a runnable main branch; integrate weekly; every member must be able to explain the full data flow.

11. Eight-Week Timeline

Week	Work
1	Requirements, feature freeze, use cases
2	Class model, file formats, wireframes, sample data
3	Models, repositories, IDs, validation
4	Stock receiving, movements, expiry alerts, FEFO

Week	Work
5	POS cart, checkout validation, invoice, allocation
6	Payments, receipts, waste module, dashboard, reports
7	Integration, backup/recovery, CSV export, full testing
8	Bug fixing, user guide, presentation, demo rehearsal

12. Key Risks

Risk	Mitigation
Scope creep	Freeze MVP; team approval for additions
Text-file inconsistency	Transaction service, journal, backups, failure tests
Late integration	Agree interfaces early; weekly merges
FEFO/date errors	Java date types, boundary tests
Money calculation errors	BigDecimal, known-total tests
Uneven contribution	Clear ownership, code reviews
Limited test time	Scripted tests; final week for stabilization

13. Expected Outcomes

- Runnable Java Swing prototype with CSV persistence
- Working product, batch, expiry, FEFO, POS, waste and report workflows
- Class diagrams, test cases, user guide and demo

Demo flow: create product with two batches > show expiry alert and FEFO order > sell across both batches > show invoice, payment and updated stock > record waste > restart app to show persistence > show insufficient-stock error and backup/restore.

14. Future Roadmap

1. **Academic MVP:** local desktop, CSV files, core features.
2. **Stronger local version:** database, roles, audit, returns, expenses.
3. **Connected version:** cloud backend, sync, multi-device.
4. **Commercial SaaS:** multi-branch, multi-tenant, subscriptions.
5. **Intelligence and integrations:** forecasting, barcode scanners, payment gateway, mobile app.

15. Conclusion

ShelfLife connects batch-level stock, expiry monitoring, FEFO allocation, billing and waste reporting in one consistent workflow. With controlled scope and priority on reliable checkout and inventory logic, it is a suitable second-semester OOP group project and leaves a clear path toward a database-backed product.

References: Oracle Java SE Documentation, Java Swing Tutorial, java.time API, Java I/O tutorial, BigDecimal API (docs.oracle.com).